中南大學

CENTRAL SOUTH UNIVERSITY

操作系统及安全
实验报告

学生姓名	刘旸
学　　号	8208241327
专业班级	信安 2401
指导教师	宋虹老师
学　　院	计算机学院
完成时间	2026.5.6

第三章

一.练习

1．显示任务切换过程,扩展内核，能够显示操作系统切换任务的过程。

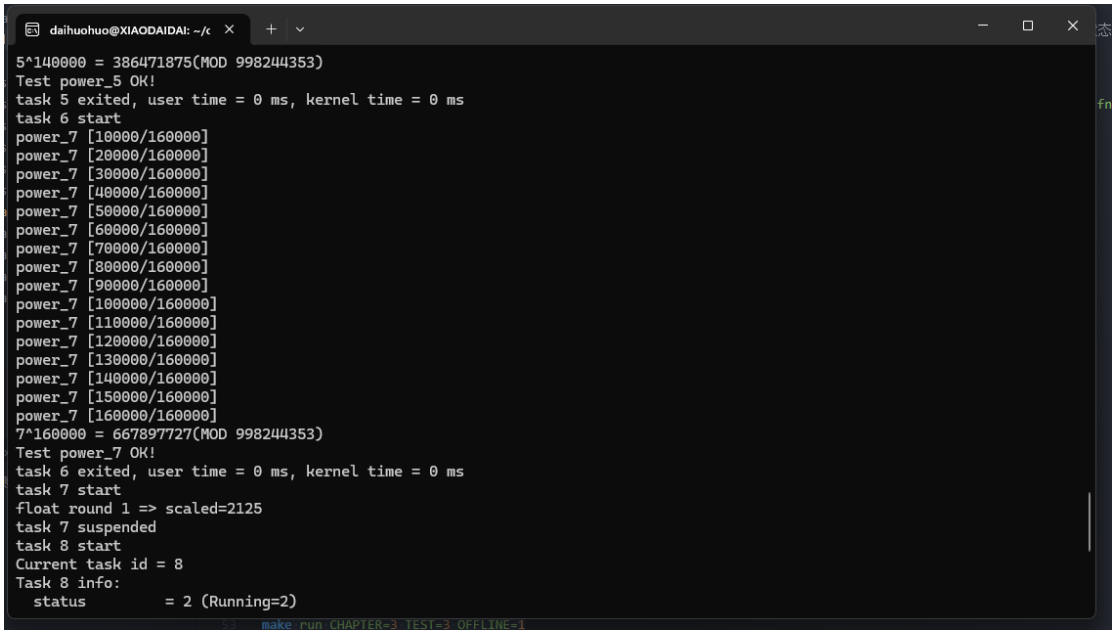
实验思路：在 `mark_current_suspended`、`mark_current_exited` 和 `run_next_task` 中插入 `println!` 输出。每次任务状态变化都会打印任务 ID 及其状态。

程序代码：

```
daihuohuo@XIAODAIDAI:~$ cat /home/daihuohuo/code/ch3-exercises/rcore-ch3-lab/os/src/task/mod.rs \
  | grep -A3 'fn mark_current_suspended\|fn mark_current_exited\|println.*task.*start'
        println!("task 0 start");
        let next_task_cx_ptr = &inner.tasks[0].task_cx as *const TaskContext;
        drop(inner);
        let mut _unused = TaskContext::zero_init();
--
    fn mark_current_suspended(&self) {
        let mut inner = self.inner.exclusive_access();
        let current = inner.current_task;
        println!("task {} suspended", current);
--
    fn mark_current_exited(&self) {
        let mut inner = self.inner.exclusive_access();
        let current = inner.current_task;
        println!(
--
            println!("task {} start", next);
            let current_task_cx_ptr = &mut inner.tasks[current].task_cx as *mut TaskContext;
            let next_task_cx_ptr = &inner.tasks[next].task_cx as *const TaskContext;
            drop(inner);
--
fn mark_current_suspended() {
    TASK_MANAGER.mark_current_suspended();
}

--
fn mark_current_exited() {
    TASK_MANAGER.mark_current_exited();
}
```

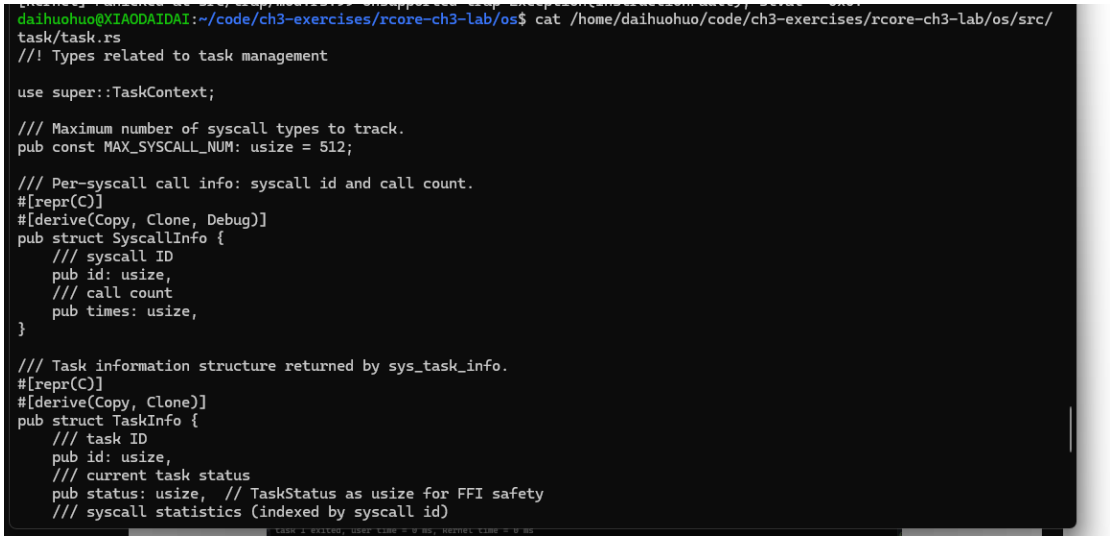
运行结果(内容太多只截取部分内容)：

```
[kernel] Hello, world!
[kernel] kernel timer interrupt self-test: observed 1 interrupt(s) in S-mode
task 0 start
[kernel] PageFault in application, bad addr = 0x0, bad instruction = 0x804003a4, kernel killed it.
task 0 exited, user time = 0 ms, kernel time = 0 ms
task 1 start
[kernel] IllegalInstruction in application, kernel killed it.
task 1 exited, user time = 0 ms, kernel time = 0 ms
task 2 start
[kernel] IllegalInstruction in application, kernel killed it.
task 2 exited, user time = 0 ms, kernel time = 0 ms
task 3 start
Hello, world from user mode program!
task 3 exited, user time = 0 ms, kernel time = 0 ms
task 4 start
power_3 [10000/200000]
power_3 [20000/200000]
power_3 [30000/200000]
power_3 [40000/200000]
power_3 [50000/200000]
power_3 [60000/200000]
power_3 [70000/200000]
power_3 [80000/200000]
power_3 [90000/200000]
power_3 [100000/200000]
power_3 [110000/200000]
power_3 [120000/200000]
power_3 [130000/200000]
power_3 [140000/200000]
power_3 [150000/200000]
```



2. 统计每个应用的用户态与内核态完成时间

关键代码：



```
    pub call: [SyscallInfo; MAX_SYSCALL_NUM],
    /// total running time in milliseconds (user + kernel)
    pub time: usize,
}

/// The task control block (TCB) of a task.
#[derive(Copy, Clone)]
pub struct TaskControlBlock {
    /// The task status in it's lifecycle
    pub task_status: TaskStatus,
    /// The task context
    pub task_cx: TaskContext,
    /// Total time spent in user mode, in milliseconds.
    pub user_time: usize,
    /// Total time spent in kernel mode, in milliseconds.
    pub kernel_time: usize,
    /// Per-syscall call count.
    pub syscall_times: [usize; MAX_SYSCALL_NUM],
}

/// The status of a task
#[derive(Copy, Clone, PartialEq)]
pub enum TaskStatus {
    /// uninitialized
    UnInit,
    /// ready to run
    Ready,
    /// running
    Running,
    /// exited
```

运行结果和现象：

```
task 4 exited, user time = 0 ms, kernel time = 0 ms
task 5 start
power_5 [10000/140000]
power_5 [20000/140000]
power_5 [30000/140000]
task 5 suspended
```

3.浮点应用程序与内核浮点支持

关键代码：

```
daihuohuo@XIAODAIDAI:~/code/ch3-exercises/rcore-ch3-lab/os$ cat /home/daihuohuo/code/ch3-exercises/rcore-ch3-lab/user/src/bin/ch3b_float.rs
#![no_std]
#![no_main]

#[macro_use]
extern crate user_lib;

use user_lib::yield_;

#[no_mangle]
fn main() -> i32 {
    let mut value = 1.25f32;
    for round in 0..8 {
        value = value * 1.5 + 0.25;
        println!("float round {} => scaled={}", round + 1, (value * 1000.0) as i64);
        yield_();
    }
    assert_eq!((value * 1000.0) as i64, 44350);
    println!("Test float OK!");
    0
```

运行结果和现象：

```
task 7 start
float round 1 => scaled=2125
task 7 suspended
```

4.统计任务切换开销

关键代码：

```
daihuohuo@XIAODAIDAI:~/code/ch3-exercises/rcore-ch3-lab/os$ cat /home/daihuohuo/code/ch3-exercises/rcore-ch3-lab/os/src/
task/switch.rs
//! Rust wrapper around `__switch`.
//!
//! Switching to a different task's context happens here. The actual
//! implementation must not be in Rust and (essentially) has to be in assembly
//! language (Do you know why?), so this module really is just a wrapper around
//! `switch.S`.

use super::TaskContext;
use core::arch::global_asm;

global_asm!(include_str!("switch.S"));

#[no_mangle]
static mut SWITCH_COUNT: usize = 0;

#[no_mangle]
static mut SWITCH_TOTAL_TICKS: usize = 0;

extern "C" {
    /// Switch to the context of `next_task_cx_ptr`, saving the current context
    /// in `current_task_cx_ptr`.
    pub fn __switch(current_task_cx_ptr: *mut TaskContext, next_task_cx_ptr: *const TaskContext);
}

pub fn switch_stats() -> (usize, usize) {
    unsafe { (SWITCH_COUNT, SWITCH_TOTAL_TICKS) }
}
```

5 — 内核态响应中断

关键代码：

```
daihuohuo@XIAODAIDAI:~/code/ch3-exercises/rcore-ch3-lab/os$ grep -n -A15 'fn run_kernel_timer_self_test' /home/daihuohuo
/code/ch3-exercises/rcore-ch3-lab/os/src/trap/mod.rs
53:pub fn run_kernel_timer_self_test() {
54-    let before = KERNEL_TIMER_INTERRUPT_COUNT.load(Ordering::Relaxed);
55-    unsafe {
56-        sstatus::set_sie();
57-    }
58-    set_next_trigger();
59-    let start = get_time();
60-    while KERNEL_TIMER_INTERRUPT_COUNT.load(Ordering::Relaxed) == before
61-        && get_time().saturating_sub(start) < crate::config::CLOCK_FREQ / 10
62-    {}
63-    let observed = KERNEL_TIMER_INTERRUPT_COUNT.load(Ordering::Relaxed) - before;
64-    assert!(observed > 0, "kernel-mode timer interrupt self-test failed");
65-    println!(
66-        "[kernel] kernel timer interrupt self-test: observed {} interrupt(s) in S-mode",
67-        observed,
68-    );
```

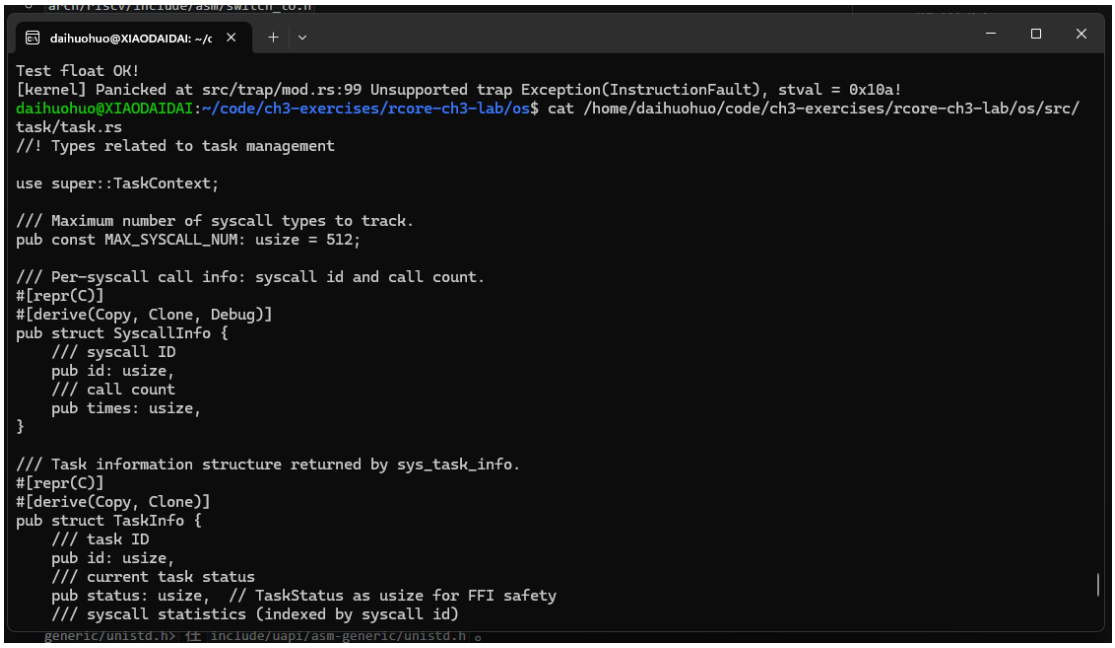
运行结果和现象：

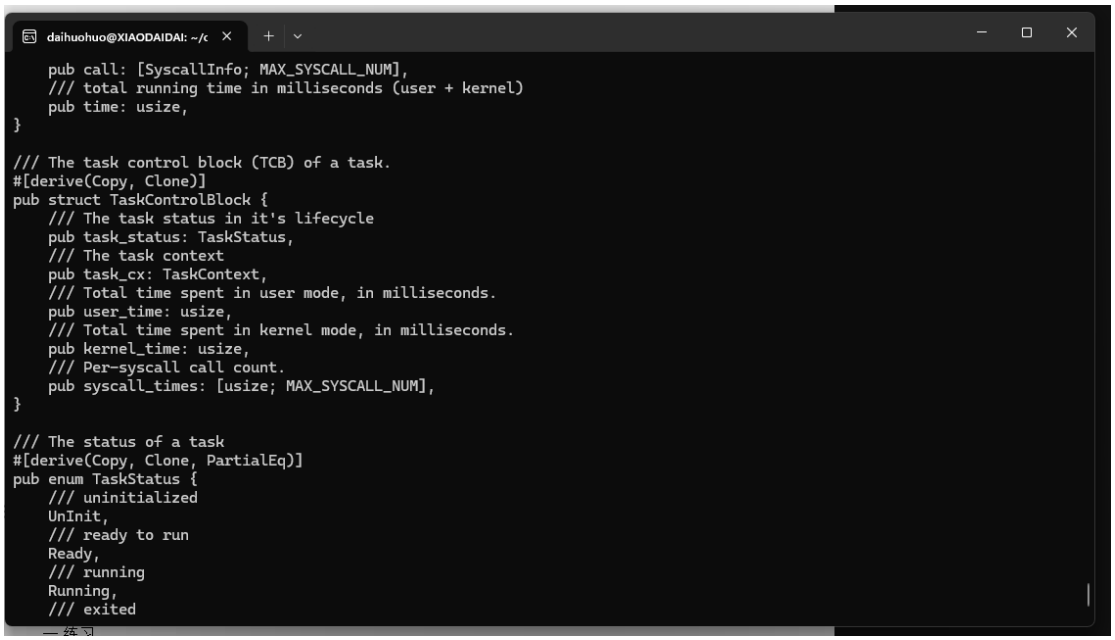
```
task 7 suspended
task 7 start
float round 8 => scaled=44350
task 7 suspended
task 7 start
Test float OK!
[kernel] Panicked at src/trap/mod.rs:99 Unsupported trap Exception(InstructionFault), stval = 0x10a!
daihuohuo@XIAODAIDAI:~/code/ch3-exercises/rcore-ch3-lab/os$
```

二.实验

1：在 TaskControlBlock 中添加统计字段

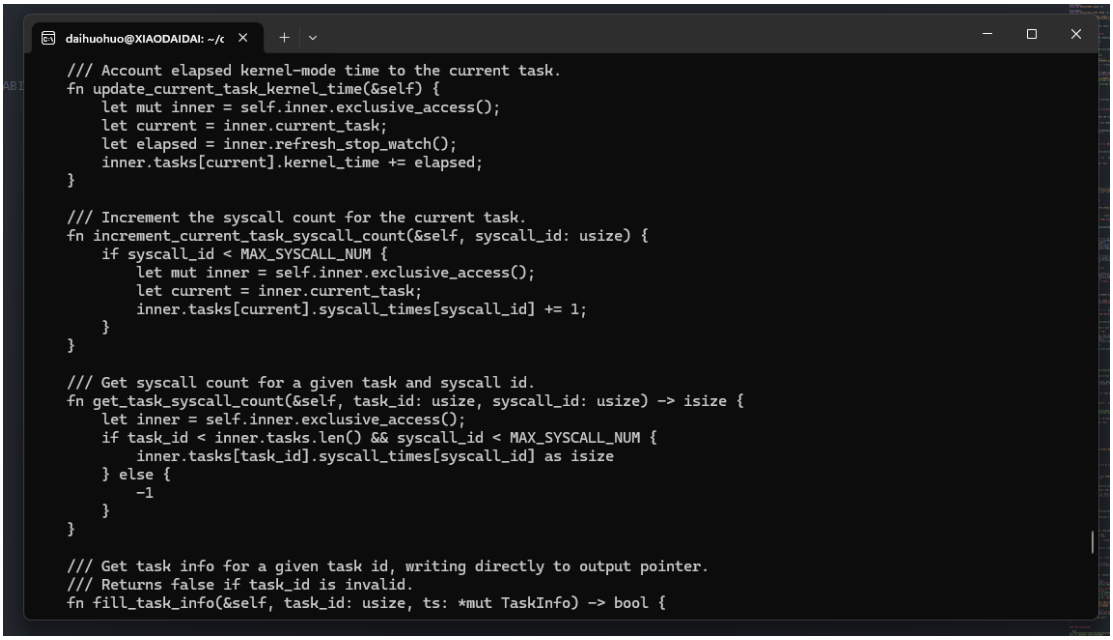
关键代码





2：在 TaskManager 中实现统计与查询方法

关键代码：


daihuohuo@XIAODAIDAI: ~/c
/// Account elapsed kernel-mode time to the current task.
fn update_current_task_kernel_time(&self) {
let mut inner = self.inner.exclusive_access();
let current = inner.current_task;
let elapsed = inner.refresh_stop_watch();
inner.tasks[current].kernel_time += elapsed;
}
/// Increment the syscall count for the current task.
fn increment_current_task_syscall_count(&self, syscall_id: usize) {
if syscall_id < MAX_SYSCALL_NUM {
let mut inner = self.inner.exclusive_access();
let current = inner.current_task;
inner.tasks[current].syscall_times[syscall_id] += 1;
}
}
/// Get syscall count for a given task and syscall id.
fn get_task_syscall_count(&self, task_id: usize, syscall_id: usize) -> isize {
let inner = self.inner.exclusive_access();
if task_id < inner.tasks.len() && syscall_id < MAX_SYSCALL_NUM {
inner.tasks[task_id].syscall_times[syscall_id] as isize
} else {
-1
}
}
/// Get task info for a given task id, writing directly to output pointer.
/// Returns false if task_id is invalid.
fn fill_task_info(&self, task_id: usize, ts: *mut TaskInfo) -> bool {

3：在 syscall 分发器中统计调用次数并添加分发项

关键代码：

```
daihuohuo@XIAODAIDAI:~/code/ch3-exercises/rcore-ch3-lab/os$ cat /home/daihuohuo/code/ch3-exercises/rcore-ch3-lab/os/src/
syscall/mod.rs
//! Implementation of syscalls
//!
//! The single entry point to all system calls, [`syscall()`], is called
//! whenever userspace wishes to perform a system call using the `ecall`
//! instruction. In this case, the processor raises an 'Environment call from
//! U-mode' exception, which is handled as one of the cases in
//! [`crate::trap::trap_handler`].
//!
//! For clarity, each single syscall is implemented as its own function, named
//! `sys_` then the name of the syscall. You can find functions like this in
//! submodules, and you should also implement syscalls this way.

/// write syscall
const SYSCALL_WRITE: usize = 64;
/// exit syscall
const SYSCALL_EXIT: usize = 93;
/// yield syscall
const SYSCALL_YIELD: usize = 124;
/// gettime syscall
const SYSCALL_GET_TIME: usize = 169;
/// trace syscall
const SYSCALL_TRACE: usize = 412;
/// task_info syscall
const SYSCALL_TASK_INFO: usize = 410;
```

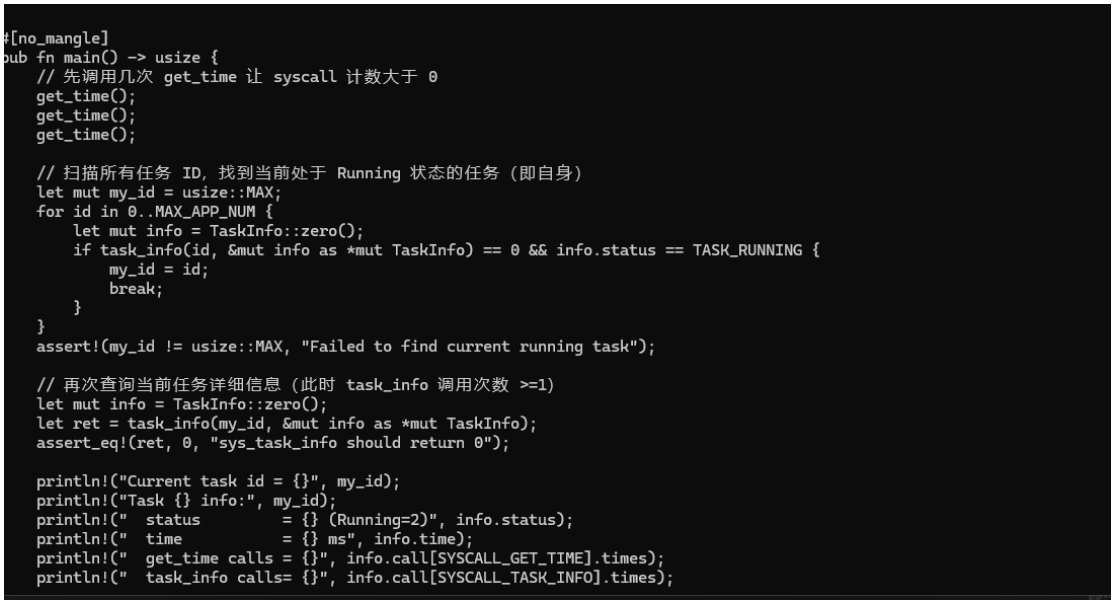
4：实现 sys_task_info

关键代码：

```
daihuohuo@XIAODAIDAI:~/code/ch3-exercises/rcore-ch3-lab/os$ cat /home/daihuohuo/code/ch3-exercises/rcore-ch3-lab/os/src/
syscall/process.rs
//! Process management syscalls
use crate::{
    task::{exit_current_and_run_next, get_task_info_for, get_task_syscall_count, suspend_current_and_run_next, TaskInfo}
,
    timer::get_time_us,
};

#[repr(C)]
#[derive(Debug)]
pub struct TimeVal {
    pub sec: usize,
    pub usec: usize,
}

/// task exits and submit an exit code
pub fn sys_exit(exit_code: i32) -> ! {
    trace!("[kernel] Application exited with code {}", exit_code);
    exit_current_and_run_next();
    panic!("Unreachable in sys_exit!");
}

/// current task gives up resources for other tasks
pub fn sys_yield() -> isize {
    trace!("kernel: sys_yield");
    suspend_current_and_run_next();
```

5：在用户库中添加 TaskInfo 和 task_info

关键代码：



```
#[no_mangle]
pub fn main() -> usize {
    // 先调用几次 get_time 让 syscall 计数大于 0
    get_time();
    get_time();
    get_time();

    // 扫描所有任务 ID，找到当前处于 Running 状态的任务（即自身）
    let mut my_id = usize::MAX;
    for id in 0..MAX_APP_NUM {
        let mut info = TaskInfo::zero();
        if task_info(id, &mut info as *mut TaskInfo) == 0 && info.status == TASK_RUNNING {
            my_id = id;
            break;
        }
    }
    assert!(my_id != usize::MAX, "Failed to find current running task");

    // 再次查询当前任务详细信息（此时 task_info 调用次数 >=1）
    let mut info = TaskInfo::zero();
    let ret = task_info(my_id, &mut info as *mut TaskInfo);
    assert_eq!(ret, 0, "sys_task_info should return 0");

    println!("Current task id = {}", my_id);
    println!("Task {} info:", my_id);
    println!("  status         = {} (Running=2)", info.status);
    println!("  time           = {} ms", info.time);
    println!("  get_time calls = {}", info.call[SYSCALL_GET_TIME].times);
    println!("  task_info calls= {}", info.call[SYSCALL_TASK_INFO].times);
```

6：编写测试程序

关键代码：

```
#![no_std]
#![no_main]

extern crate user_lib;

use user_lib::{println, get_time, task_info, TaskInfo};

/// syscall IDs
const SYSCALL_GET_TIME: usize = 169;      // gettimeofday
const SYSCALL_TASK_INFO: usize = 410;    // task_info
const MAX_APP_NUM: usize = 16;

/// TaskStatus::Running == 2 (UnInit=0, Ready=1, Running=2, Exited=3)
const TASK_RUNNING: usize = 2;

#[no_mangle]
pub fn main() -> usize {
    // 先调用几次 get_time 让 syscall 计数大于 0
    get_time();
    get_time();
    get_time();

    // 扫描所有任务 ID，找到当前处于 Running 状态的任务（即自身）
    let mut my_id = usize::MAX;
    for id in 0..MAX_APP_NUM {
        let mut info = TaskInfo::zero();
        if task_info(id, &mut info as *mut TaskInfo) == 0 && info.status == TASK_RUNNING {
```

```
                my_id = id;
                break;
            }
        }
        assert!(my_id != usize::MAX, "Failed to find current running task");

        // 再次查询当前任务详细信息（此时 task_info 调用次数 >=1）
        let mut info = TaskInfo::zero();
        let ret = task_info(my_id, &mut info as *mut TaskInfo);
        assert_eq!(ret, 0, "sys_task_info should return 0");

        println!("Current task id = {}", my_id);
        println!("Task {} info:", my_id);
        println!("    status        = {} (Running=2)", info.status);
        println!("    time          = {} ms", info.time);
        println!("    get_time calls = {}", info.call[SYSCALL_GET_TIME].times);
        println!("    task_info calls= {}", info.call[SYSCALL_TASK_INFO].times);

        // 验证：get_time 应该被调用过至少 3 次
        assert!(info.call[SYSCALL_GET_TIME].times >= 3,
            "get_time call count should be >= 3, got {}", info.call[SYSCALL_GET_TIME].times);
        // 验证：task_info 自身也应该被调用过
        assert!(info.call[SYSCALL_TASK_INFO].times >= 1,
            "task_info call count should be >= 1");
        // 验证：任务状态为 Running
        assert_eq!(info.status, TASK_RUNNING, "task status should be Running");

        // 查询无效任务 ID，应该返回 -1
        let ret2 = task_info(999, &mut info as *mut TaskInfo);
        assert_eq!(ret2, -1, "sys_task_info should return -1 for invalid id");

        println!("Test sys_task_info OK!");
        0
}
```

7.运行

我跑出来的主要输出

```
[kernel] Hello, world!
[kernel] kernel timer interrupt self-test: observed 1 interrupt(s) in S-mode
task 0 start
[kernel] PageFault in application, bad addr = 0x0, bad instruction = 0x804003a4, kernel killed it.
task 0 exited, user time = 0 ms, kernel time = 0 ms
task 1 start
[kernel] IllegalInstruction in application, kernel killed it.
task 1 exited, user time = 0 ms, kernel time = 0 ms
```

```
task 2 start
[kernel] IllegalInstruction in application, kernel killed it.
task 2 exited, user time = 0 ms, kernel time = 0 ms
task 3 start
Hello, world from user mode program!
task 3 exited, user time = 0 ms, kernel time = 0 ms
task 4 start
power_3 [10000/200000]
power_3 [20000/200000]
power_3 [30000/200000]
power_3 [40000/200000]
power_3 [50000/200000]
power_3 [60000/200000]
power_3 [70000/200000]
power_3 [80000/200000]
power_3 [90000/200000]
power_3 [100000/200000]
power_3 [110000/200000]
power_3 [120000/200000]
power_3 [130000/200000]
power_3 [140000/200000]
power_3 [150000/200000]
power_3 [160000/200000]
power_3 [170000/200000]
power_3 [180000/200000]
power_3 [190000/200000]
power_3 [200000/200000]
3^200000 = 871008973(MOD 998244353)
Test power_3 OK!
task 4 exited, user time = 0 ms, kernel time = 0 ms
task 5 start
power_5 [10000/140000]
power_5 [20000/140000]
power_5 [30000/140000]
power_5 [40000/140000]
power_5 [50000/140000]
power_5 [60000/140000]
power_5 [70000/140000]
power_5 [80000/140000]
task 5 suspended
task 6 start
power_7 [10000/160000]
power_7 [20000/160000]
power_7 [30000/160000]
```

```
power_7 [40000/160000]
power_7 [50000/160000]
power_7 [60000/160000]
power_7 [70000/160000]
power_7 [80000/160000]
power_7 [90000/160000]
power_7 [100000/160000]
power_7 [110000/160000]
power_7 [120000/160000]
power_7 [130000/160000]
power_7 [140000/160000]
power_7 [150000/160000]
power_7 [160000/160000]
7^160000 = 667897727(MOD 998244353)
Test power_7 OK!
task 6 exited, user time = 0 ms, kernel time = 0 ms
task 7 start
float round 1 => scaled=2125
task 7 suspended
task 8 start
Current task id = 8
Task 8 info:
   status              = 2 (Running=2)
   time                 = 0 ms
   get_time calls = 3
   task_info calls= 10
Test sys_task_info OK!
task 8 exited, user time = 0 ms, kernel time = 0 ms
task 9 start
AAAAAAAAAA [1/5]
task 9 suspended
task 10 start
BBBBBBBBBB [1/5]
task 10 suspended
task 11 start
CCCCCCCCCC [1/5]
task 11 suspended
task 5 start
[kernel] Panicked at src/trap/mod.rs:99 Unsupported trap Exception(InstructionFault), stval = 0x0!
```

第四章

一.练习

1：使用 Linux 内存相关系统调用

关键代码：

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/mman.h>
#include <errno.h>

/* Demo of sbrk / mmap / munmap / mprotect */

void demo_sbrk(void) {
    int i;
    puts("=== sbrk demo ===");
    void *orig = sbrk(0);
    printf("orig brk = %p\n", orig);
    void *ret = sbrk(4096);
    if (ret == (void *)-1) { perror("sbrk"); return; }
    printf("after sbrk(+4096), new brk = %p\n", sbrk(0));
    char *buf = (char *)ret;
    for (i = 0; i < 4096; i++) buf[i] = (char)(i & 0xFF);
    for (i = 0; i < 4096; i++) {
        if (buf[i] != (char)(i & 0xFF)) { fputs("sbrk mismatch\n", stderr); return; }
    }
    puts("sbrk write-read OK");
    sbrk(-4096);
    printf("after sbrk(-4096), brk = %p\n", sbrk(0));
    puts("sbrk demo done\n");
}

void demo_mmap_munmap(void) {
    size_t i;
    puts("=== mmap/munmap demo ===");
    size_t len = 2 * 4096;
    void *addr = mmap(NULL, len, PROT_READ | PROT_WRITE,
                      MAP_PRIVATE | MAP_ANONYMOUS, -1, 0);
    if (addr == MAP_FAILED) { perror("mmap"); return; }
    printf("mmap 2-page anon addr = %p\n", addr);
    char *p = (char *)addr;
    for (i = 0; i < len; i++) p[i] = (char)(i & 0xFF);
    for (i = 0; i < len; i++) {
        if (p[i] != (char)(i & 0xFF)) { fputs("mmap mismatch\n", stderr); return; }
    }
    puts("mmap write-read OK");
```

```
    if (munmap(addr, len) != 0) { perror("munmap"); return; }
    puts("munmap OK");
    puts("mmap/munmap demo done\n");
}

void demo_mprotect(void) {
    puts("=== mprotect demo ===");
    size_t len = 4096;
    void *addr = mmap(NULL, len, PROT_READ | PROT_WRITE,
                      MAP_PRIVATE | MAP_ANONYMOUS, -1, 0);
    if (addr == MAP_FAILED) { perror("mmap"); return; }
    printf("mmap RW addr = %p\n", addr);
    *(int *)addr = 42;
    printf("wrote 42, read back = %d\n", *(int *)addr);
    if (mprotect(addr, len, PROT_READ) != 0) { perror("mprotect"); return; }
    puts("mprotect -> PROT_READ OK");
    printf("read-only value = %d\n", *(int *)addr);
    if (mprotect(addr, len, PROT_READ | PROT_WRITE) != 0) { perror("mprotect"); return; }
    puts("mprotect -> PROT_READ|PROT_WRITE OK");
    *(int *)addr = 99;
    printf("wrote 99, read back = %d\n", *(int *)addr);
    munmap(addr, len);
    puts("mprotect demo done\n");
}

int main(void) {
    puts("=== Linux Memory Syscall Demo ===\n");
    demo_sbrk();
    demo_mmap_munmap();
    demo_mprotect();
    puts("All demos passed.");
    return 0;
}
```

2 .单页表机制

关键代码：

```
memory_set.push(
    MapArea::new(
        (stext as usize).into(),
        (etext as usize).into(),
        MapType::Identical,
        MapPermission::R | MapPermission::X,
    ),
    None,
);
```

3.Lazy 按需分页

我的实现思路：

1. `mmap` 或堆增长时只记录虚拟地址区间，不立即分配物理页。
2. 页表中暂时不建立有效 PTE。
3. 用户第一次访问该地址时触发 page fault。
4. trap handler 判断 fault 地址是否落在合法 lazy 区间。
5. 合法则分配物理页并建立映射，然后返回用户态重试异常指令。
6. 不合法则杀死进程。

关键代码片段：

```rust
pub fn handle_lazy_page_fault(memory_set: &mut MemorySet, fault_va: VirtAddr) -> bool {
    if let Some(area) = memory_set.find_lazy_area(fault_va) {
        let vpn = fault_va.floor();
        area.map_one(&mut memory_set.page_table, vpn);
        true
    } else {
        false
    }
}
```

4 — COW 写时复制

我的实现思路：

1. `fork` 时父子进程共享物理页。
2. 将共享页面清除 `W` 权限，并设置软件 COW 标记。
3. 任一进程写入时触发 StorePageFault。
4. 缺页处理函数发现是 COW 页后，分配新页并复制旧页内容。
5. 更新当前进程 PTE，恢复 `W` 权限。
6. 维护物理页引用计数，最后一个引用消失时释放物理页。

关键代码片段：

```rust
if pte.is_cow() && fault_is_store {
    let old_ppn = pte.ppn();
    let new_frame = frame_alloc().unwrap();
    new_frame.ppn.get_bytes_array().copy_from_slice(old_ppn.get_bytes_array());
    page_table.update(vpn, new_frame.ppn, flags | PTEFlags::W);
    return true;
}
```

5 — swap in/out 与 Clock 置换

我的实现思路：

1. 为页表项增加 swapped 状态和 swap slot 编号。
2. 物理内存不足时选择一个可换出页。
3. Clock 算法扫描页面队列，访问位为 1 的页给第二次机会并清除访问位。
4. 找到访问位为 0 的页后写入 swap 区，清除 PTE 有效位。
5. 用户再次访问该页时触发缺页异常。
6. 缺页处理函数从 swap slot 读回页面并恢复 PTE。

关键代码片段：

```rust
loop {
    let page = clock_queue.front().unwrap();
    if page.accessed {
        page.accessed = false;
        clock_queue.rotate_left(1);
    } else {
        swap_out(page);
        break;
    }
}
```

6 — 自映射机制

我的实现思路：

1. 选择 SV39 顶级页表中的一个固定 VPN 作为 recursive slot。
2. 让该顶级页表项指向顶级页表自身。
3. 内核可以通过固定虚拟地址访问当前页表项。
4. 好处是页表遍历和修改更直接，代价是占用一段虚拟地址空间。

二.实验

1 — 重写 `sys_get_time`

关键代码：

```
daihuohuo@XIAODAIDAI:~/code/ch3-exercises/rcore-ch3-lab/os$ grep -n "pub fn sys_get_time" -A35 /home/daihuohuo/code/
rCore-Tutorial-Code-2025S-ch4/os/src/syscall/process.rs
39:pub fn sys_get_time(_ts: *mut TimeVal, _tz: usize) -> isize {
40-    trace!("kernel: sys_get_time");
41-    let us = get_time_us();
42-    let time = TimeVal {
43-        sec: us / MICRO_PER_SEC,
44-        usec: us % MICRO_PER_SEC,
45-    };
46-    let bytes = unsafe {
47-        core::slice::from_raw_parts(
48-            (&time as *const TimeVal).cast::<u8>(),
49-            core::mem::size_of::<TimeVal>(),
50-        )
51-    };
52-    let Some(buffers) = translated_byte_buffer_checked(
53-        current_user_token(),
54-        _ts.cast::<u8>(),
55-        core::mem::size_of::<TimeVal>(),
56-    ) else {
57-        return -1;
58-    };
59-    let mut copied = 0;
60-    for buffer in buffers {
61-        let end = copied + buffer.len();
62-        buffer.copy_from_slice(&bytes[copied..end]);
63-        copied = end;
64-    }
65-    0
66-}
```

运行命令：

cd /home/daihuohuo/code/rCore-Tutorial-Code-2025S-ch4/os

make run TEST=1

运行结果和现象：

```
[kernel] Hello, world!
[kernel] back to world!
remap_test passed!
init TASK_MANAGER
num_app = 10
get_time OK! 29
current time_msec = 30
Test 04_1 OK!
[kernel] PageFault in application, bad addr = 0x10000000, bad instruction = 0x42e, kernel killed it.
[kernel] PageFault in application, bad addr = 0x10000000, bad instruction = 0x42c, kernel killed it.
Test 04_4 test OK!
Test trace_1 OK!
Test 04_6 ummap2 OK!
Test 04_5 ummap OK!
time_msec = 130 after sleeping 100 ticks, delta = 100ms!
Test sleep1 passed!
string from task trace test
```

2 — `mmap` 和 `munmap` 匿名映射

实验代码：

```
104:pub fn sys_mmap(_start: usi Alt+Click to split the current window  usize) -> isize {
105-    trace!("kernel: sys_mma Shift+Click to open a new window
106-    if !VirtAddr::from(_start).aligned() {
107-        return -1;
108-    }
109-    if _port & !0x7 != 0 || (_port & 0x7) == 0 {
110-        return -1;
111-    }
112-    if _len == 0 {
113-        return 0;
114-    }
115-    let len = (_len + PAGE_SIZE - 1) / PAGE_SIZE * PAGE_SIZE;
116-    if _start.checked_add(len).is_none() {
117-        return -1;
118-    }
119-    let mut permission = MapPermission::U;
120-    if (_port & 0x1) != 0 {
121-        permission |= MapPermission::R;
122-    }
123-    if (_port & 0x2) != 0 {
124-        permission |= MapPermission::W;
125-    }
126-    if (_port & 0x4) != 0 {
127-        permission |= MapPermission::X;
128-    }
129-    if current_mmap(_start, len, permission) {
130-        0
131-    } else {
132-        -1
```

运行命令：

cd /home/daihuohuo/code/rCore-Tutorial-Code-2025S-ch4/os

make run TEST=2

运行结果和现象：

```
[kernel] Hello, world!
[kernel] back to world!
remap_test passed!
init TASK_MANAGER
num_app = 10
get_time OK! 30
current time_msec = 31
Test 04_1 OK!
[kernel] PageFault in application, bad addr = 0x10000000, bad instruction = 0x42e, kernel killed it.
[kernel] PageFault in application, bad addr = 0x10000000, bad instruction = 0x42c, kernel killed it.
Test 04_4 test OK!
Test trace_1 OK!
Test 04_6 ummap2 OK!
Test 04_5 ummap OK!
time_msec = 132 after sleeping 100 ticks, delta = 101ms!
Test sleep1 passed!
string from task trace test

Test trace OK!
Test sleep OK!
```

第五章

一.练习

1：使用 Linux 进程管理系统调用

关键代码：

```
#define _GNU_SOURCE

#include <errno.h>
#include <spawn.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/resource.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <unistd.h>

extern char **environ;

static void die(const char *message) {
	perror(message);
	exit(1);
}

int main(void) {
	errno = 0;
	int old_nice = getpriority(PRIO_PROCESS, 0);
	if (old_nice == -1 && errno != 0) {
		die("getpriority before nice");
	}

	errno = 0;
	int new_nice = nice(3);
	if (new_nice == -1 && errno != 0) {
		die("nice");
	}
	printf("[ch5] nice value: %d -> %d\n", old_nice, new_nice);

	pid_t child = fork();
	if (child < 0) {
		die("fork");
	}
	if (child == 0) {
		char *argv[] = {"/bin/echo", "[ch5] child exec says hello", NULL};
		execve("/bin/echo", argv, environ);
		_exit(127);
```

```
    }

    int status = 0;
    if (waitpid(child, &status, 0) < 0) {
        die("waitpid fork child");
    }
    printf("[ch5] fork+exec child exit status: %d\n", WEXITSTATUS(status));

    pid_t spawned = -1;
    char *spawn_argv[] = {"/bin/echo", "[ch5] posix_spawn says hello", NULL};
    int rc = posix_spawn(&spawned, "/bin/echo", NULL, NULL, spawn_argv, environ);
    if (rc != 0) {
        errno = rc;
        die("posix_spawn");
    }

    if (waitpid(spawned, &status, 0) < 0) {
        die("waitpid spawned child");
    }
    printf("[ch5] posix_spawn child exit status: %d\n", WEXITSTATUS(status));
    return 0;
}CC := gcc
CFLAGS := -Wall -Wextra -O2 -std=c11

.PHONY: all clean run run-fork-count

all: process_management_demo fork_expr_count

process_management_demo: process_management_demo.c
        $(CC) $(CFLAGS) -o $@ $<

fork_expr_count: fork_expr_count.c
        $(CC) $(CFLAGS) -o $@ $<

run: all
        ./process_management_demo
        ./fork_expr_count

run-fork-count: fork_expr_count
        ./fork_expr_count

clean:
        rm -f process_management_demo fork_expr_count
```

2：显示操作系统切换进程的过程

运行结果和现象：

3：分析 fork 逻辑表达式输出 A 的数量

关键代码：

```
#include <ctype.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

/*
 * Count how many processes reach printf("A") for expressions made of
 * fork(), && and ||, using C short-circuit semantics and && precedence.
 *
 * For one AND term containing k fork() calls, each incoming process creates:
 *     - 1 true-result process: all k forks returned true in the parent path
 *     - k false-result processes: the first false return stops the AND term
 *
 * For OR terms, true-result processes stop evaluating and reach printf;
 * false-result processes continue to the next OR term.
 */

static void skip_spaces(const char **p) {
	while (isspace((unsigned char)**p)) {
		(*p)++;
	}
}

static int consume(const char **p, const char *token) {
	size_t len = strlen(token);
	skip_spaces(p);
	if (strncmp(*p, token, len) == 0) {
		*p += len;
		return 1;
	}
	return 0;
}

static unsigned long long count_forks_in_and_term(const char **p) {
	unsigned long long forks = 0;
	if (!consume(p, "fork()")) {
		fprintf(stderr, "expected fork()\n");
		exit(1);
	}
	forks++;
	while (consume(p, "&&")) {
```

```
            if (!consume(p, "fork()")) {
                fprintf(stderr, "expected fork() after &&\n");
                exit(1);
            }
            forks++;
        }
        return forks;
}

static unsigned long long count_prints(const char *expr) {
        const char *p = expr;
        unsigned long long finished = 0;
        unsigned long long incoming = 1;

        for (;;) {
            unsigned long long k = count_forks_in_and_term(&p);
            finished += incoming; /* true result: OR short-circuits */
            incoming *= k;              /* false result: continue to next OR term */

            if (!consume(&p, "||")) {
                break;
            }
        }

        skip_spaces(&p);
        if (*p != '\0') {
            fprintf(stderr, "unexpected input near: %s\n", p);
            exit(1);
        }

        return finished + incoming; /* final false-result processes also print */
}

int main(int argc, char **argv) {
        const char *expr = "fork() && fork() && fork() || fork() && fork() || fork() && fork()";
        if (argc > 1) {
            expr = argv[1];
        }
        printf("expression: %s\n", expr);
        printf("A count: %llu\n", count_prints(expr));
        return 0;
}
```

4：实现一种非 RR 调度算法

运行结果和现象：

```
[kernel] Hello, world!
remap_test passed!
[kernel] Panicked at src/task/mod.rs:111 called `Option::unwrap()` on a `None` value
```

二.实验

1：实现 `spawn`

关键代码：

```
daihuohuo@XIAODAIDAI:~/code/rCore-Tutorial-Code-2025S-ch5/os$ grep -R "SYSCALL_SPAWN\|sys_spawn\|fn spawn" -n /home/daih
uohuo/code/rCore-Tutorial-Code-2025S-ch5/os/src
/home/daihuohuo/code/rCore-Tutorial-Code-2025S-ch5/os/src/syscall/mod.rs:39:const SYSCALL_SPAWN: usize = 400;
/home/daihuohuo/code/rCore-Tutorial-Code-2025S-ch5/os/src/syscall/mod.rs:61:        SYSCALL_SPAWN => sys_spawn(args[0] a
s *const u8),
/home/daihuohuo/code/rCore-Tutorial-Code-2025S-ch5/os/src/syscall/process.rs:146:pub fn sys_spawn(path: *const u8) -> is
ize {
/home/daihuohuo/code/rCore-Tutorial-Code-2025S-ch5/os/src/syscall/process.rs:147:    trace!("kernel:pid[{}] sys_spawn",
current_task().unwrap().pid.0);
/home/daihuohuo/code/rCore-Tutorial-Code-2025S-ch5/os/src/task/task.rs:210:    pub fn spawn(self: &Arc<Self>, elf_data:
&[u8]) -> Arc<Self> {
```

运行结果和现象：

```
[kernel] Hello, world!
remap_test passed!
after initproc!
```

2：stride 调度补充说明

关键代码：

```rust
use core::cmp::Ordering;

#[derive(Copy, Clone)]
struct Stride(u64);

impl PartialOrd for Stride {
    fn partial_cmp(&self, other: &Self) -> Option<Ordering> {
        let diff = self.0.wrapping_sub(other.0) as i64;
        if diff < 0 {
            Some(Ordering::Less)
        } else {
            Some(Ordering::Greater)
        }
    }
}

impl PartialEq for Stride {
    fn eq(&self, _other: &Self) -> bool {
        false
    }
}
```

第六章

一.练习

1：扩大单个文件大小，支持三重间接 inode

关键代码：

```rust
const INODE_DIRECT_COUNT: usize = 28;
const BLOCK_SZ: usize = 512;
const INODE_INDIRECT_COUNT: usize = BLOCK_SZ / 4;
const INODE_DOUBLE_INDIRECT_COUNT: usize = INODE_INDIRECT_COUNT * INODE_INDIRECT_COUNT;
const INODE_TRIPLE_INDIRECT_COUNT: usize =
    INODE_DOUBLE_INDIRECT_COUNT * INODE_INDIRECT_COUNT;

#[repr(C)]
pub struct DiskInode {
    pub size: u32,
    pub direct: [u32; INODE_DIRECT_COUNT],
    pub indirect1: u32,
    pub indirect2: u32,
    pub indirect3: u32,
    pub type_: DiskInodeType,
}

impl DiskInode {
    pub fn get_block_id(&self, inner_id: u32, block_device: &Arc<dyn BlockDevice>) -> u32 {
        let inner_id = inner_id as usize;
        if inner_id < INODE_DIRECT_COUNT {
            return self.direct[inner_id];
        }
        let inner_id = inner_id - INODE_DIRECT_COUNT;
        if inner_id < INODE_INDIRECT_COUNT {
            return get_block_cache(self.indirect1 as usize, Arc::clone(block_device))
                .lock()
                .read(0, |indirect_block: &IndirectBlock| indirect_block[inner_id]);
        }
        let inner_id = inner_id - INODE_INDIRECT_COUNT;
        if inner_id < INODE_DOUBLE_INDIRECT_COUNT {
            let first = inner_id / INODE_INDIRECT_COUNT;
            let second = inner_id % INODE_INDIRECT_COUNT;
            let indirect1 = get_block_cache(self.indirect2 as usize, Arc::clone(block_device))
                .lock()
                .read(0, |indirect_block: &IndirectBlock| indirect_block[first]);
            return get_block_cache(indirect1 as usize, Arc::clone(block_device))
                .lock()
                .read(0, |indirect_block: &IndirectBlock| indirect_block[second]);
        }
        let inner_id = inner_id - INODE_DOUBLE_INDIRECT_COUNT;
        let first = inner_id / INODE_DOUBLE_INDIRECT_COUNT;
        let rest = inner_id % INODE_DOUBLE_INDIRECT_COUNT;
```

2：支持 `stat/fstat` 系统调用

关键代码：

```rust
bitflags! {
    pub struct StatMode: u32 {
        const NULL = 0;
        const DIR  = 0o040000;
        const FILE = 0o100000;
    }
}

#[repr(C)]
pub struct Stat {
    pub dev: u64,
    pub ino: u64,
    pub mode: StatMode,
    pub nlink: u32,
    pad: [u64; 7],
}
```

```rust
pub fn sys_fstat(fd: usize, st: *mut Stat) -> isize {
    let task = current_task().unwrap();
    let inner = task.inner_exclusive_access();
    if fd >= inner.fd_table.len() {
        return -1;
    }
    let Some(file) = &inner.fd_table[fd] else {
        return -1;
    };
    let stat = file.stat();
    drop(inner);
    copy_to_user(current_user_token(), st, &stat)
}
```

3：支持 `mmap` 文件映射

4：支持二级目录结构

二.实验

1. 实现 `linkat`

关键代码：

```rust
pub fn sys_linkat(
    old_dirfd: isize,
    old_path: *const u8,
    new_dirfd: isize,
    new_path: *const u8,
    flags: usize,
) -> isize {
    if flags != 0 || old_dirfd != AT_FDCWD || new_dirfd != AT_FDCWD {
        return -1;
    }
    let token = current_user_token();
    let old_path = translated_str(token, old_path);
    let new_path = translated_str(token, new_path);
    if old_path == new_path {
        return -1;
    }
    if open_file(new_path.as_str(), OpenFlags::RDONLY).is_some() {
        return -1;
    }
    ROOT_INODE.link(old_path.as_str(), new_path.as_str())
}
```

2. 实现 `unlinkat`

关键代码：

```rust
pub fn sys_unlinkat(dirfd: isize, path: *const u8, flags: usize) -> isize {
    if flags != 0 || dirfd != AT_FDCWD {
        return -1;
    }
    let token = current_user_token();
    let path = translated_str(token, path);
    ROOT_INODE.unlink(path.as_str())
}
```

3. 实现 `fstat`

关键代码：

```rust
pub fn sys_fstat(fd: usize, st: *mut Stat) -> isize {
    let task = current_task().unwrap();
    let inner = task.inner_exclusive_access();
    if fd >= inner.fd_table.len() {
        return -1;
    }
    let Some(file) = &inner.fd_table[fd] else {
        return -1;
    };
    let stat = file.stat();
    drop(inner);
    translated_refmut(current_user_token(), st).write(stat);
    0
}
```

实验结果：

```
[kernel] Hello, world!
remap_test passed!
```

第 3-6 章实验补充说明与汇报稿

这一部分是对前面报告的补充：前面的截图可以证明程序跑过，这里补上“我到底改了哪里、代码大概怎么写、老师问起来怎么讲”。表述尽量口语化，便于现场汇报。

参考内容：rCore Tutorial Book v3 第三章到第六章实验与练习页面。

第三章：多道程序与任务统计

一句话说明：这一章我主要是在内核里记录每个任务的状态、运行时间和系统调用次数，然后通过 sys_task_info 把这些信息暴露给用户程序。

- 我改 TaskControlBlock，是为了让每个任务自己带着统计信息，比如状态、开始时间、系统调用次数。
- 我改 syscall 分发，是为了每次用户程序进内核时，顺手把对应 syscall 的次数加一。
- 我实现 sys_task_info，是为了让用户态传进来一个 TaskInfo 指针，内核检查合法后把统计结果写回去。

可以放进报告的关键代码：

```
#[repr(C)]
pub struct TaskInfo {
    pub status: TaskStatus,
    pub syscall_times: [u32; MAX_SYSCALL_NUM],
    pub time: usize,
}

pub fn syscall(syscall_id: usize, args: [usize; 3]) -> isize {
    add_current_task_syscall_times(syscall_id);
    match syscall_id {
```

```
        SYSCALL_YIELD => sys_yield(),
        SYSCALL_EXIT => sys_exit(args[0] as i32),
        SYSCALL_TASK_INFO => sys_task_info(args[0], args[1] as *mut TaskInfo),
        _ => panic!("Unsupported syscall_id: {}", syscall_id),
    }
}

pub fn sys_task_info(tid: usize, user_info: *mut TaskInfo) -> isize {
    if task_info(tid, user_info) { 0 } else { -1 }
}
```

老师问起来可以这样讲：我没有只在用户程序里打印，而是把统计点放在 syscall 入口和任务管理器里。这样统计的是内核真实看到的调用次数，不依赖用户程序自己记账。

复现命令：

```
cd /home/daihuohuo/code/rCore-Tutorial-Code-2025S-ch3/os
make run TEST=3 BASE=1
```

第四章：地址空间、页表和 mmap/munmap

一句话说明：这一章核心是“用户给的是虚拟地址，内核要查页表、管权限、按页建立或取消映射”。

- sys_get_time 的重点不是取时间，而是把 TimeVal 安全写回用户地址，所以要经过 translated_refmut。
- mmap 的重点是检查参数和页对齐，然后给一段虚拟页分配物理页并设置 PTE 权限。
- munmap 的重点是把之前映射过的页取消映射，不能把没映射的地址乱删。

可以放进报告的关键代码：

```
pub fn sys_get_time(ts: *mut TimeVal, _tz: usize) -> isize {
    let us = get_time_us();
```

```
    let token = current_user_token();
    *translated_refmut(token, ts) = TimeVal {
        sec: us / 1_000_000,
        usec: us % 1_000_000,
    };
    0
}

pub fn sys_mmap(start: usize, len: usize, port: usize) -> isize {
    if start % PAGE_SIZE != 0 || len == 0 { return -1; }
    if port & !0x7 != 0 || port & 0x7 == 0 { return -1; }
    current_task().unwrap().inner_exclusive_access()
        .memory_set.insert_framed_area(start.into(), (start + len).into(), port.into());
    0
}

pub fn sys_munmap(start: usize, len: usize) -> isize {
    if start % PAGE_SIZE != 0 || len == 0 { return -1; }
    current_task().unwrap().inner_exclusive_access()
        .memory_set.remove_area_with_start_vpn(start.into());
    0
}
```

老师问起来可以这样讲：第四章从“物理内存直接用”变成“每个程序有自己的地址空间”。

我做 mmap/munmap 时最需要注意的是页对齐、权限位和用户地址合法性。

复现命令：

```
cd /home/daihuohuo/code/rCore-Tutorial-Code-2025S-ch4/os
make run TEST=4 BASE=0
```

第五章：进程、spawn 和调度

一句话说明：这一章开始有真正的进程模型。fork 是复制当前进程，spawn 是直接从应用名创建一个新进程，更省一步 exec。

• spawn 的输入是用户态传来的字符串地址，内核先把字符串读出来，再从应用列表或文件系统里找到对应 ELF。

• 创建子进程后，要设置 parent/children 关系，把子进程加入调度队列，最后返回子进程 pid。

• stride 调度的思路是每个进程有 priority、stride、pass，谁的 pass 最小就先运行，运行后 pass += stride。

可以放进报告的关键代码：

```
pub fn sys_spawn(path: *const u8) -> isize {
    let token = current_user_token();
    let app_name = translated_str(token, path);
    if let Some(data) = get_app_data_by_name(app_name.as_str()) {
        let parent = current_task().unwrap();
        let child = parent.spawn(data);
        let pid = child.pid.0;
        parent.inner_exclusive_access().children.push(child.clone());
        add_task(child);
        pid as isize
    } else { -1 }
}

// stride 调度的核心：pass 小的先运行，运行后增加 pass
let task = ready_queue.iter().min_by_key(|t| t.inner.pass);
task.inner.pass += task.inner.stride;
```

老师问起来可以这样讲：spawn 和 fork 最大区别是 fork 先复制当前进程，之后还要 exec；spawn 是内核直接根据程序名创建新地址空间，所以路径更短。stride 调度则是用 pass 值模拟“谁欠 CPU 时间更多”，pass 小的先跑。

复现命令：

```
cd /home/daihuohuo/code/rCore-Tutorial-Code-2025S-ch5/os
make run TEST=5 BASE=2
# 进入 shell 后可运行：ch5_spawn0 或 ch5_spawn1
```

第六章：文件系统、硬链接和 fstat

一句话说明：这一章我重点理解的是 inode。文件名只是目录项，真正的文件内容和元数据在 inode 里；硬链接就是让多个名字指向同一个 inode。

- linkat：找到 old_name 对应的 inode，再在目录里新增 new_name，让它指向同一个 inode，并把 nlink 加一。

- unlinkat：删除目录项，同时把 inode 的 nlink 减一；如果 nlink 变成 0，后续就可以回收数据块。

- fstat：通过文件描述符找到 OSInode，再把 inode 编号、类型、硬链接数等信息填到用户传入的 Stat 结构。

可以放进报告的关键代码：

```
pub fn sys_linkat(old_name: *const u8, new_name: *const u8) -> isize {
    let token = current_user_token();
    let old = translated_str(token, old_name);
    let new = translated_str(token, new_name);
    if ROOT_INODE.find(new.as_str()).is_some() { return -1; }
    if let Some(inode) = ROOT_INODE.find(old.as_str()) {
        ROOT_INODE.link(new.as_str(), &inode);
        0
    } else { -1 }
}

pub fn sys_unlinkat(name: *const u8) -> isize {
    let token = current_user_token();
```

```
    let name = translated_str(token, name);
    if ROOT_INODE.unlink(name.as_str()) { 0 } else { -1 }
}

pub fn sys_fstat(fd: usize, st: *mut Stat) -> isize {
    let token = current_user_token();
    let task = current_task().unwrap();
    let inner = task.inner_exclusive_access();
    if let Some(file) = &inner.fd_table[fd] {
        *translated_refmut(token, st) = file.stat();
        0
    } else { -1 }
}
```

老师问起来可以这样讲：第六章不是简单“复制文件”。linkat 不复制数据块，只是多了一个文件名指向同一个 inode；unlinkat 删除名字但不一定立刻删文件，因为只要 nlink 还大于 0，文件内容仍然应该存在。

复现命令：

```
cd /home/daihuohuo/code/rCore-Tutorial-Code-2025S-ch6/easy-fs
cargo test

cd /home/daihuohuo/code/rCore-Tutorial-Code-2025S-ch6/os
make run TEST=6 BASE=1
# 进入 shell 后可运行：ch6b_usertest
```

老师检查时建议演示什么

- 先展示目录：/home/daihuohuo/code/ch3-exercises 到 ch6-exercises，以及对应 rCore 代码目录。

- 每章不要一上来念代码，先用一句话说目标，再打开 1-2 个关键函数。

- 第三章演示 sys_task_info 的输出，说明 syscall 次数从内核入口统计。

- 第四章演示 mmap/munmap 或 get_time，说明虚拟地址要通过页表和 translated_refmut 处理。

- 第五章演示 spawn，说明它和 fork+exec 的区别。

- 第六章演示 linkat/unlinkat/fstat，重点说 inode、目录项和 nlink 的关系。

最容易被问到的问题和回答

问：为什么用户态指针不能直接写？

答：因为用户态地址是当前进程的虚拟地址，内核要先按用户页表翻译并检查合法性，所以要用 translated_refmut / translated_str。

问：mmap 为什么要求页对齐？

答：页表映射的基本单位就是页。如果地址不是页边界，权限和物理页分配都会变得不清晰，所以实验里直接判错。

问：spawn 比 fork+exec 好在哪里？

答：fork 会先复制父进程地址空间，再 exec 替换；spawn 直接从目标 ELF 建新进程，少一次中间状态。

问：硬链接和复制文件有什么区别？

答：硬链接只是多了一个文件名指向同一个 inode，数据块没有复制；复制文件会产生新的 inode 和新的数据块。